

Devops

Antonio Espín Herranz

Contenidos

- Organización de proyectos C++ modernos
- Gestión del ciclo de vida del microservicio
- Integración con pipelines de CI/CD (conceptual)

Devops

- **DevOps** es un conjunto de prácticas, cultura y herramientas que integran **Desarrollo (Dev)** y **Operaciones (Ops)** para automatizar todo el ***ciclo de vida*** del software:
 - Desde el código hasta la producción.
- Su objetivo es:
 - **Reducir el tiempo** entre escribir código y tenerlo funcionando en producción
 - **Aumentar la calidad** mediante automatización
 - **Eliminar tareas manuales** que generan errores
 - **Hacer despliegues frecuentes y seguros**

Organización de proyectos C++ modernos

- Normalmente se van a organizar en múltiples proyectos dentro de una solución:

/solución

/app	→ ejecutable (UI, CLI, servicio...)
/core	→ librería estática con la lógica de negocio
/utils	→ librería estática con utilidades comunes
/thirdparty	→ dependencias externas (vcpkg)
/tests	→ proyecto de tests (GoogleTest)
/docs	→ documentación

Organización de proyectos C++ modernos

- **1. Proyecto app (EXE)**
 - **Responsabilidad:** solo orquestar, no tener lógica.
 - main.cpp
 - Configuración, parseo de argumentos, DI (si usas)
 - Llama a funciones de core
 - **No debe contener:**
 - Algoritmos
 - Lógica de negocio
 - Acceso a datos

Organización de proyectos C++ modernos

- **2. Proyecto core (Static Library .lib)**
 - **Responsabilidad:** toda la lógica de negocio.
 - Clases principales
 - Modelos
 - Algoritmos
 - Servicios internos
 - **Ventajas:**
 - Reutilizable
 - Testeable
 - Independiente del ejecutable

Organización de proyectos C++ modernos

- **3. Proyecto utils (Static Library .lib)**

- **Responsabilidad:** utilidades comunes.

- Ejemplos:

- Logging
 - Helpers de fechas
 - Validaciones
 - Wrappers de sistema

Organización de proyectos C++ modernos

- **4. Proyecto tests (GoogleTest)**
 - **Responsabilidad:** pruebas unitarias.
 - Referencia a core y utils
 - No referencia a app

Organización de proyectos C++ modernos

- **5. Dependencias externas (vcpkg)**
 - **Responsabilidad:** gestión de librerías externas.
 - Ejemplos:
 - fmt
 - spdlog
 - nlohmann-json
 - sqlite/mysql connector
 - boost

Organización de proyectos C++ modernos

- **Cómo se conectan los proyectos en Visual Studio**
 - En el **Explorador de soluciones**:
 - Clic derecho en app → **Add** → **Reference**
 - Marca:
 - core
 - utils
 - En tests:
 - Referencia core
 - Referencia utils
 - **Nunca** referencias app desde tests.

Organización de proyectos C++ modernos

- Configurar **MSBuild**:
 - En cada proyecto indicar la versión de C++
 - Por ejemplo: **std:c++20**
 - Nivel Warning: **/W4**
 - Tratar warning como errores: **/WX**
 - **Sanitizers (VS 2022)**
 - AddressSanitizer
 - UndefinedBehaviorSanitizer
 - **Precompiled headers (PCH)**
 - Para acelerar builds.

Organización de proyectos C++ modernos

- **Cuándo usar CMake en vez de MSBuild**

- Si el proyecto:
 - Es multiplataforma
 - Usa muchas dependencias externas
 - Va a compilarse en CI/CD Linux
 - Va a compilarse en WSL
- Entonces → CMake.
- Si es **solo Windows**, MSBuild es perfecto.

Resumen

Proyecto	Tipo	Contenido	Referencia a
app	EXE	main, UI, CLI	core, utils
core	Static Library	lógica de negocio	utils
utils	Static Library	helpers, logging	—
tests	GoogleTest	pruebas unitarias	core, utils
thirdparty	vcpkg	dependencias	todos

Gestión del ciclo de vida del microservicio

- **Diseño:** qué hace y qué no hace.
- **Implementación:** código + tests + contratos claros.
- **Build:** artefacto reproducible (imagen, binario).
- **Despliegue:** ejecución gestionada (K8s/VM).
- **Observabilidad:** logs, métricas, trazas.
- **Evolución/retirada:** cambios seguros, versionado, deprecación.

Diseño

- **Objetivo:** decidir qué hace y qué NO hace.
 - **Bounded context: Definir el límite funcional:** un microservicio debe resolver un problema concreto (p.ej. Billing, Users, Inventory), no ser “misc”.
 - **Contratos y API:**
 - Diseñar la API pública (REST/gRPC/Eventos).
 - Definir modelos de datos expuestos (DTOs) y errores.
 - **Responsabilidades internas:**
 - Lógica de negocio.
 - Integraciones externas (otros servicios, DB, colas).
 - Reglas de validación, seguridad, auditoría.

Implementación

- **Objetivo:** construir el servicio de forma aislada, pero integrable.
- **Estructura del proyecto:**
 - Código del servicio (controladores, dominio, infraestructura).
 - Tests unitarios y de integración.
 - Configuración (variables de entorno, secrets, conexiones).
- **Buenas prácticas técnicas:**
 - Manejo de errores consistente.
 - Timeouts y reintentos en llamadas externas.
 - Circuit breakers, si aplica.
 - Logs estructurados (JSON, niveles, correlación).
- **Pruebas locales:**
 - Tests unitarios.
 - Tests de integración contra DB/colas en contenedores (Docker Compose).
 - Simulación de dependencias (mocks, stubs).

Empaquetado y construcción → Build

- **Objetivo:** generar un artefacto desplegable y reproducible.
 - **Build del servicio:**
 - Compilar (C++).
 - Ejecutar tests.
 - Generar binario o paquete.
 - **Contenerización:**
 - Crear Dockerfile con:
 - Imagen base mínima.
 - Copia del binario/config.
 - Usuario no root.
 - **Registro de artefactos:**
 - Subir imagen a un registry (Docker Hub, ACR, ECR...).
 - Versionado semántico (v1.3.0, etc.).

Despliegue

- **Objetivo:** ejecutar el servicio en un entorno gestionado (**Kubernetes**).
 - **Despliegue:**
 - Kubernetes: Deployment, Service, Ingress, ConfigMap, Secret.
 - **Configuración en runtime:**
 - Variables de entorno (conexiones, feature flags).
 - Secrets (tokens, passwords, certificados).
 - **Descubrimiento y comunicación:**
 - Service discovery (DNS, service mesh).
 - Balanceo de carga.
 - Autenticación/autorización entre servicios (mTLS, JWT, API Gateway).

Observabilidad

- **Objetivo:** entender qué hace el servicio y cómo se comporta.
- **Logging:**
 - Logs estructurados.
 - Correlation IDs para seguir una petición entre servicios.
- **Métricas:**
 - Latencia, throughput
 - Recursos: CPU, memoria, conexiones.
- **Tracing distribuido:**
 - OpenTelemetry, Jaeger
 - Ver el recorrido de una petición a través de múltiples microservicios.
- **Health checks:**
 - liveness (¿el proceso está vivo?).
 - readiness (¿está listo para recibir tráfico?).
- **Mantenimiento:**
 - Parcheo de librerías.
 - Actualización de runtime.
 - Ajuste de recursos (autoscaling).

Evolución / Retirada

- **Objetivo:** cambiar sin romper a los consumidores.
 - **Evolución de la API:**
 - Versionado (/v1, /v2) o compatibilidad hacia atrás.
 - Deprecación controlada de endpoints/campos.
 - **Despliegues progresivos:**
 - Blue/Green, Canary, Feature flags.
 - Rollback rápido si algo va mal.
 - **Retirada del servicio:**
 - Aviso a consumidores.
 - Redirección a nuevo servicio o versión.
 - Eliminación de despliegues, reglas de routing y datos asociados (si procede).

Integración con pipelines de CI/CD (conceptual)

- Un **pipeline de CI/CD** es una **cadena automatizada de pasos** que se ejecutan cada vez que haces un cambio en el código de un proyecto.
 - Sirve para:
 - **Construir** el proyecto
 - **Probarlo**
 - **Empaquetarlo**
 - **Desplegarlo**
 - **Monitorearlo**
 - Todo **sin intervención humana**.

CI / CD

- **CI — *Continuous Integration***

- **Integración continua.**

- Su objetivo es:

- Validar que el código nuevo **no rompe nada**
 - Ejecutar tests automáticamente
 - Compilar el proyecto
 - Analizar calidad y seguridad

- **CI responde a la pregunta:**

- “¿Mi código es correcto y estable?”

- **CD — *Continuous Delivery / Continuous Deployment***

- **Entrega o despliegue continuo.**

- Su objetivo es:

- Crear artefactos listos para producción (binarios, imágenes Docker)
 - Desplegar automáticamente en entornos (dev, QA, staging, prod)
 - Hacer rollouts seguros (canary, blue/green)
 - Automatizar rollback si algo falla

- **CD responde a la pregunta:**

- “¿Mi software está listo y puede desplegarse automáticamente?”

Los pipelines

- Sin **pipelines**:
 - Cada desarrollador compila “a su manera”
 - Los **tests** se olvidan
 - Los despliegues son manuales y propensos a errores
 - No hay trazabilidad
 - No hay velocidad
- Con **pipelines**:
 - Todo es **repetible**
 - Todo es **automático**
 - Todo es **seguro**
 - Todo es **rápido**

Fases de un pipeline

- Compilación
- Test
- Análisis Estadístico
- Empaquetado
- Despliegue
- Cada fase se ejecuta automáticamente cuando hacemos:
 - Un commit
 - Un merge
 - Un tag de una versión

Herramientas

- No existe “una única herramienta”, sino **un stack mínimo** que cubre:
 - Compilación
 - Tests
 - Análisis estático
 - Empaquetado (Docker)
 - Despliegue
 - Observabilidad

Herramientas C++ en CI

- **CMake** — estándar de facto para builds multiplataforma
- **MSBuild** — si trabajas solo en Windows/Visual Studio
- **vcpkg** — gestor de dependencias oficial de Microsoft
- **Ninja** — backend rápido para CMake

Herramientas de Test automáticos

- **GoogleTest** — framework estándar de testing
- **CTest** — ejecutor de tests integrado con CMake
- **Catch2** — alternativa ligera

Herramientas de análisis estático y calidad

- **clang-tidy** — análisis estático moderno
- **cppcheck** — análisis estático clásico
- **clang-format** — formateo automático
- **sanitizers** — detección de errores en runtime

Herramientas para empaquetado y distribución

- **Docker** — empaquetar el microservicio en una imagen
- **Dockerfile_multistage** — compilar y empaquetar en un solo flujo
- **OCI_registry** — subir la imagen (ACR, ECR, **Docker Hub**)

Herramientas para despliegue automático

- **Kubernetes_manifests** — YAML de despliegue
- **Helm** — plantillas para despliegues complejos
- **kubectl** — aplicar cambios desde el pipeline

Herramientas para observabilidad

- **Prometheus** — métricas
- **Grafana** — dashboards
- **OpenTelemetry** — trazas distribuidas
- **ELK_stack** — logs estructurados

Herramientas para ejecutar el pipeline

- **GitHub_Actions**
- **Azure_DevOps**
- **GitLab_CI**
- **Jenkins**
- Estas herramientas **no compilan C++**, sino que **orquestan** las herramientas anteriores.

Resumen

Fase	Herramientas
Build	CMake, MSBuild, Ninja, vcpkg
Tests	GoogleTest, CTest
Calidad	clang-tidy, cppcheck, sanitizers
Package	Docker, Dockerfile multistage
Deploy	Kubernetes, Helm, kubectl
CI/CD	GitHub Actions / Azure DevOps / GitLab CI